



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

AI-Based Multi-Label Food Spoilage Detection Using Image Analysis and IoT

An Interdisciplinary Project Report (XX367P)

Submitted by,

A J Deeksha

1RV23BT001

G Daksha Reddy

1RV23BT024

Sanika Santosh Paithankar

1RV24CD404

Harish Appasab Savalagi

1RV24AI403

Under the guidance of

Prof. Mekhala Vinod Purohit

Assistant Professor

Dept. of CSE

RV College of Engineering®

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in respective departments

2025-26



RV College of Engineering[®], Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)



CERTIFICATE

Certified that the interdisciplinary project (XX367P) work titled ***AI-Based Multi-Label Food Spoilage Detection Using Image Analysis and IoT*** is carried out by of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in respective departments during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the interdisciplinary project report deposited in the departmental library. The interdisciplinary project report has been approved as it satisfies the academic requirements in respect of interdisciplinary project work prescribed by the institution for the said degree.

Prof. Mekhala Vinod Purohit
Guide

Dr. Ramakanth Kumar P
Head of the Department

Dr. M.V. Renukadevi
Dean Academics

Dr. K. N. Subramanya
Principal

External Viva

Name of Examiners

Signature with Date

- 1.
- 2.

DECLARATION

We, **A J Deeksha, G Daksha Reddy, Sanika Santosh Paithankar, Harish Appasab Savalagi** students of sixth semester B.E., RV College of Engineering, Bengaluru, hereby declare that the interdisciplinary project titled '**AI-Based Multi-Label Food Spoilage Detection Using Image Analysis and IoT**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in respective departments during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. A J Deeksha(1RV23BT001)
2. G Daksha Reddy(1RV23BT024)
3. Sanika Santosh Paithankar(1RV24CD404)
4. Harish Appasab Savalagi(1RV24AI403)

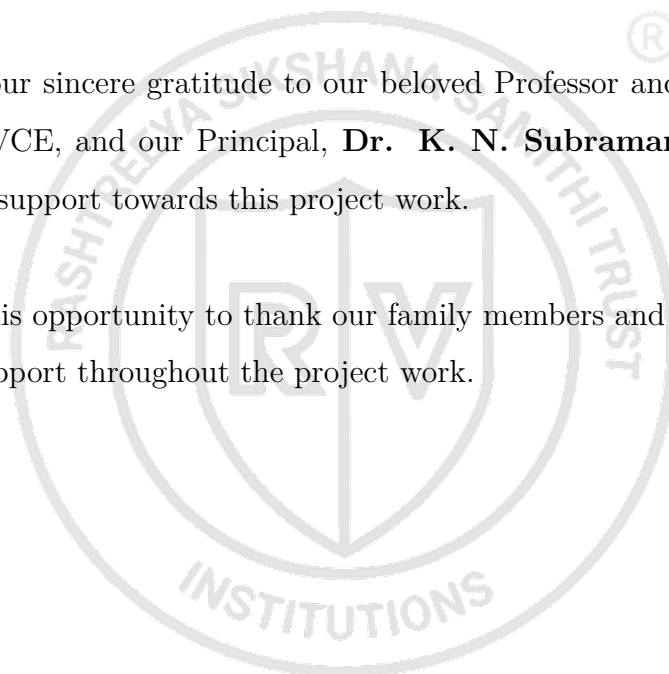
ACKNOWLEDGEMENTS

We are deeply indebted to our guide, **Prof. Mekhala Vinod Purohit**, Assistant Professor, Department of Computer Science and Engineering, RVCE, for her wholehearted support, suggestions, and invaluable advice throughout our project work and her assistance in the preparation of this report.

We express our sincere gratitude to **Dr. M.V. Renukadevi**, Professor and Dean Academics, RVCE, for the continuous support in the successful execution of this Interdisciplinary project.

We also express our sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE, and our Principal, **Dr. K. N. Subramanya**, RVCE, for their appreciation and support towards this project work.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.



ABSTRACT

Food spoilage is a critical issue that affects both health and the economy. Conventional methods for food quality monitoring are often slow and subjective, necessitating more advanced and automated solutions. This project presents an AI-based multi-label food spoilage detection system using image analysis and IoT technology.

The primary objective of this work is to develop a deep learning system capable of categorizing fruits and vegetables into four stages: Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage. The system employs multiple deep learning architectures, including CNN, MobileNetV2, ResNet50, and EfficientNetB0, for image classification. Furthermore, it integrates IoT sensors such as MQ-135 and DHT11/DHT22 for real-time monitoring of gas concentration, temperature, and humidity, which significantly influence the spoilage process.

A key feature of the proposed system is the implementation of Explainable AI (XAI) using Grad-CAM, which provides visual heatmaps to interpret the model's classification decisions. A web-based interface has been developed to provide real-time spoilage prediction, environmental data monitoring, and visualization. Preliminary results demonstrate high accuracy in classification across the four stages, and the integrated IoT platform enables continuous monitoring, thereby assisting in food waste reduction and ensuring food safety.

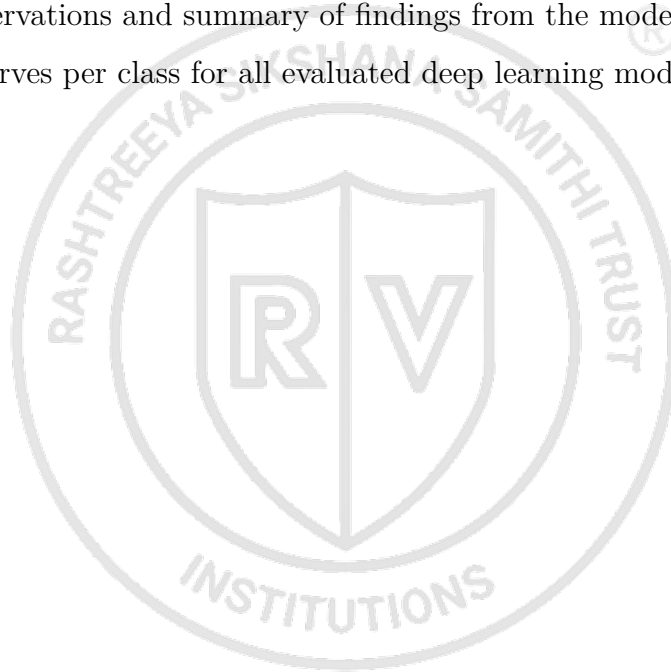
CONTENTS

Abstract	i
List of Figures	iv
List of Tables	v
1 AI-Based Multi-Label Food Spoilage Detection Using Image Analysis and IoT	1
1.1 Introduction	2
1.2 Literature Review	3
1.3 Motivation	5
1.4 Problem statement	5
1.5 Objectives	6
1.6 Brief Methodology of the project	6
1.7 Assumptions made / Constraints of the project	7
2 THEORY AND FUNDAMENTALS OF AI-BASED FOOD SPOILAGE DETECTION	8
2.1 Fundamentals of Food Spoilage	9
2.2 Image Processing and Deep Learning for Spoilage Detection	10
2.3 IoT Sensors and Environmental Monitoring	11
2.4 Explainable AI and Sensor Fusion	11
3 DATASET PREPARATION AND PRE-ANALYSIS	13
3.1 Dataset Collection and Organization	14
3.2 Spoilage Stage Labelling Strategy	14
3.3 Image Pre-processing Techniques	15
3.4 Data Augmentation Methods	15
3.5 Dataset Splitting and Balancing	16
3.6 Tools, Frameworks, and Software Used	16
3.7 Hardware and IoT Components Used	17
4 METHODOLOGY AND IMPLEMENTATION WORKFLOW	18
4.1 Overall Methodology	19
4.2 Deep Learning Workflow	20
4.3 IoT Data Acquisition and Sensor Integration	21

4.4	Explainable AI and Prediction System	22
4.5	Detailed Hardware Configuration	22
4.5.1	Hardware Components and Specifications	23
4.5.2	Comprehensive Wiring and Connection Reference	23
4.5.3	Sensor Calibration and Predictive Logic Thresholds	24
4.5.4	Blynk Cloud Configuration and Automation	24
4.5.5	Power Management and System Initialization Sequence	26
5	DEEP LEARNING AND IoT-BASED SPOILAGE DETECTION SYSTEM	28
5.1	System Architecture	29
5.2	Deep Learning Model Development	30
5.3	IoT-Based Environmental Monitoring	31
5.4	Sensor Fusion and Prediction System	32
5.5	Explainable AI Using Grad-CAM	32
5.6	Streamlit-Based Web Interface	33
6	RESULTS AND DISCUSSION	34
6.1	Dataset Distribution and Visualization	35
6.2	Model Training and Validation Performance	35
6.3	Confusion Matrix and Classification Analysis	37
6.4	ROC Curve and Per-Class Performance Analysis	39
7	CONCLUSION AND FUTURE SCOPE	41
7.1	Future Scope	43
7.2	Learning Outcomes of the Project	43
	APPENDIX	44
A.1	Importing Required Libraries	44
A.2	Dataset Preprocessing and Augmentation	45
A.3	MobileNetV2 Model Architecture	45
A.4	Model Compilation and Training	46
A.5	Prediction Function	46
A.6	Grad-CAM Visualization	47
A.7	Streamlit Web Interface	47
A.8	Performance Evaluation Metrics	48
A.9	IoT Firmware: ESP8266 Blynk Integration	48
	Bibliography	55

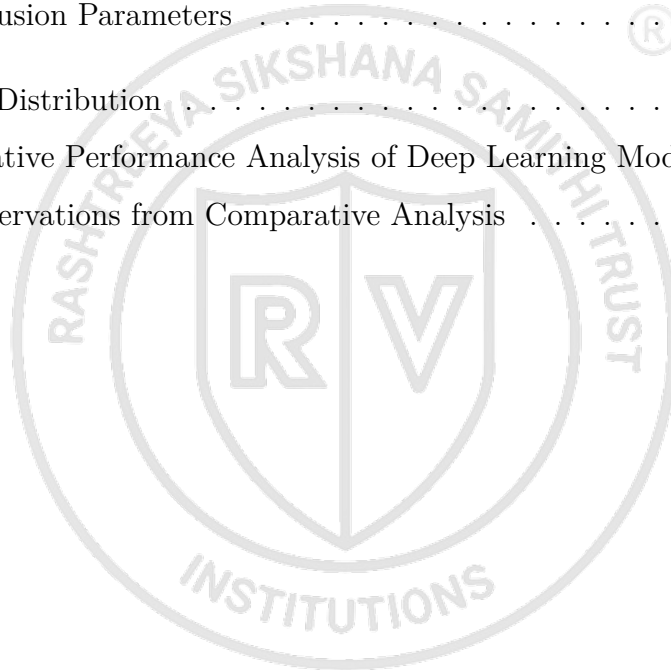
LIST OF FIGURES

4.1	Flowchart of the proposed AI-based multi-label food spoilage detection and IoT fusion methodology.	20
6.1	Final comparative performance analysis of deep learning models including training time and model size.	37
6.2	Model comparison across all metrics on the same test set.	37
6.3	Confusion Matrices for CustomCNN, MobileNetV2, ResNet50, and EfficientNetB0 on the same test set.	38
6.4	Key observations and summary of findings from the model evaluation. . .	39
6.5	ROC Curves per class for all evaluated deep learning models.	40



LIST OF TABLES

4.1	Methodology Workflow of the Proposed System	20
4.2	IoT Sensors Used in the Project	22
4.3	Complete Hardware Wiring Reference for NodeMCU ESP8266	23
4.4	Sensor Thresholds and Spoilage Stage Classification	24
5.1	Major Components of the Proposed System	30
5.2	Deep Learning Models Used	31
5.3	IoT Hardware Components	31
5.4	Sensor Fusion Parameters	32
6.1	Dataset Distribution	35
6.2	Comparative Performance Analysis of Deep Learning Models	37
6.3	Key Observations from Comparative Analysis	39





Chapter 1

AI-Based Multi-Label Food Spoilage Detection Using Image Analysis and IoT

CHAPTER 1

AI-BASED MULTI-LABEL FOOD SPOILAGE DETECTION USING IMAGE ANALYSIS AND IOT

Food spoilage is a significant challenge in the global food supply chain, leading to economic losses and health risks. Conventional spoilage detection methods are often time-consuming and subjective. Recent advances in artificial intelligence and Internet of Things (IoT) offer promising solutions for real-time and objective food quality monitoring. This project focuses on developing a multi-label classification system for food spoilage detection using image analysis and IoT integration. This chapter provides the background, literature review, objectives, methodology, and problem statement for the proposed study.

1.1 Introduction

Food spoilage is one of the major global problems affecting food quality, public health, and economic sustainability. Large quantities of fruits and vegetables are wasted every year due to microbial growth, environmental conditions, and improper storage practices. Traditional methods used for spoilage detection mainly rely on manual inspection based on visual appearance, smell, and texture changes. These approaches are subjective, inconsistent, and unsuitable for large-scale real-time monitoring applications. With the rapid advancement of Artificial Intelligence (AI), Deep Learning, Computer Vision, and Internet of Things (IoT) technologies, automated food quality monitoring systems have become an important area of research. AI-based image analysis techniques can identify spoilage indicators such as discoloration, mold formation, dark spots, and texture degradation more accurately and consistently than manual inspection methods.

This project proposes an AI-based multi-label food spoilage detection system using image analysis and IoT integration. Unlike conventional systems that classify food only as fresh or rotten, the proposed system categorizes fruits and vegetables into four spoilage stages: Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage. Deep learning models such as CNN, MobileNetV2, ResNet50, and EfficientNetB0 are used for image classification, while IoT sensors such as MQ-135 and DHT11/DHT22 are used to moni-

tor environmental factors including gas concentration, temperature, and humidity. The system also incorporates Explainable AI using Grad-CAM visualization to improve interpretability and transparency of predictions. A simple web-based interface is developed to provide real-time spoilage prediction and visualization, helping reduce food waste and improve food quality monitoring.

1.2 Literature Review

Several researchers have explored the application of Artificial Intelligence, Deep Learning, Computer Vision, and Internet of Things (IoT) technologies for food spoilage detection and food quality monitoring. Conventional food inspection methods mainly depend on manual observation, which is often subjective, time-consuming, and inconsistent. To overcome these limitations, researchers have developed automated image-based systems capable of detecting spoilage indicators such as discoloration, mold formation, bruises, texture degradation, and surface damage in fruits and vegetables. Convolutional Neural Networks (CNNs) are among the most widely used approaches for food image classification because of their strong feature extraction and pattern recognition capabilities. Various studies have shown that CNN-based systems can achieve high classification accuracy for identifying spoiled food products under controlled conditions.

Recent advancements in transfer learning have further improved spoilage detection performance. Researchers have used pretrained architectures such as MobileNetV2, ResNet50, VGG16, EfficientNet, and DenseNet for feature extraction and classification tasks. These models are trained on large datasets such as ImageNet and fine-tuned for food spoilage applications, reducing training time and improving prediction accuracy even with limited datasets. Gupta et al. proposed an AI-based spoilage detection system using CNN models integrated with IoT sensors and ESP32 modules for real-time monitoring. Their work demonstrated that combining image analysis with environmental sensing improved detection reliability and enabled early spoilage prediction. Similarly, Mulla et al. developed a multimodal spoilage detection framework combining image processing and sensor data for improved food quality monitoring.

Researchers have also investigated advanced hybrid AI systems capable of performing multiple tasks simultaneously, including spoilage classification, freshness prediction, and shelf-life estimation. Transformer-based architectures combined with CNN and Long Short-Term Memory (LSTM) networks have shown promising performance in multi-task

food analysis systems. Some studies used optical imaging and microscopy-based approaches combined with AI algorithms for bacterial detection and microbial spoilage analysis. These systems demonstrated high accuracy but often required expensive laboratory setups and controlled environments, limiting their practical implementation.

IoT-based monitoring systems have gained significant attention in recent years because environmental conditions play an important role in food deterioration. Temperature, humidity, and gas concentration influence microbial growth and spoilage progression. Several researchers integrated gas sensors such as MQ-135, ammonia sensors, and VOC sensors with microcontrollers such as Arduino, Raspberry Pi, and ESP32 for environmental monitoring. Sensor data collected from these devices were combined with machine learning algorithms to predict spoilage conditions and generate alerts. ESP32-based architectures are particularly popular because of their low cost, wireless connectivity, and ease of implementation in smart monitoring systems.

Explainable Artificial Intelligence (XAI) techniques have also become important in food spoilage detection research. Most deep learning systems operate as black-box models and do not provide information regarding how predictions are generated. To improve transparency and interpretability, researchers have used Grad-CAM and heatmap-based visualization methods to highlight image regions influencing classification results. These techniques help users understand whether the model focuses on meaningful spoilage indicators such as mold patches, dark spots, or texture variations during prediction.

Although existing research demonstrates significant progress in AI-based food spoilage detection, several limitations still remain. Most systems perform only binary classification such as fresh versus rotten and do not provide detailed information regarding spoilage progression. Many studies rely on small or non-diverse datasets, reducing the generalization capability of the models in real-world environments. Several systems also lack integration of environmental sensor data, real-time deployment capability, and Explainable AI features. Prototype-level implementations dominate current research, and practical user-friendly applications for household or small-scale food monitoring are still limited. Therefore, there is a need for a comprehensive system combining multi-label spoilage classification, IoT sensor fusion, Explainable AI, and real-time web-based deployment for efficient food quality monitoring and food waste reduction.

1.3 Motivation

Food spoilage is a major global issue that causes significant food waste and health risks. Fruits and vegetables are highly perishable and can spoil rapidly due to microbial growth, environmental conditions, and improper storage practices. Traditional spoilage detection methods mainly rely on manual inspection based on visual appearance, smell, and texture, which are often subjective, inconsistent, and time-consuming. These methods are not suitable for large-scale or real-time food quality monitoring applications.

Recent advancements in Artificial Intelligence (AI), Deep Learning, Computer Vision, and Internet of Things (IoT) technologies have enabled the development of automated food monitoring systems. AI-based image analysis techniques can identify spoilage indicators such as discoloration, mold formation, dark spots, and texture degradation more accurately than manual methods. IoT sensors can additionally monitor environmental factors such as temperature, humidity, and gas concentration, which strongly influence spoilage conditions.

Most existing food spoilage detection systems mainly perform binary classification such as fresh or rotten and provide limited information regarding spoilage progression. Many systems also lack real-time applicability, explainability, and environmental sensing integration. Therefore, this project is motivated by the need to develop a low-cost, intelligent, and user-friendly system capable of performing multi-label spoilage classification using AI and IoT technologies. The project also focuses on improving transparency and prediction reliability through Explainable AI and sensor fusion techniques for practical food quality monitoring and food waste reduction.

1.4 Problem statement

Food spoilage leads to significant food waste, economic loss, and health risks due to the consumption of deteriorated food products. Traditional spoilage detection methods mainly rely on manual inspection based on visual appearance, odor, and texture changes, which are subjective, inconsistent, and unsuitable for real-time monitoring applications. Existing AI-based food spoilage detection systems primarily perform binary classification such as fresh or rotten and do not provide detailed information regarding the progression of spoilage stages. Many existing approaches also rely only on image analysis and fail to incorporate environmental factors such as temperature, humidity, and gas concentration, which play a major role in food deterioration.

Although deep learning models have shown promising results in image classification tasks, many current systems still lack Explainable AI integration, practical real-time implementation, and multimodal sensor fusion capability. In addition, limited datasets and lack of stage-wise spoilage classification reduce the effectiveness and real-world applicability of existing systems. Therefore, there is a need for an AI-based multi-label food spoilage detection system capable of classifying fruits and vegetables into Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage stages using image analysis along with IoT-based environmental sensing. The proposed system aims to integrate deep learning, sensor fusion, Explainable AI, and a user-friendly interface to improve spoilage prediction accuracy, transparency, and practical food quality monitoring.

1.5 Objectives

The objectives of the project are:

1. To develop an AI-based system capable of classifying fruits and vegetables into multiple spoilage stages such as Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage using deep learning techniques.
2. To integrate IoT-based environmental sensing using temperature, humidity, and gas sensors for improving spoilage detection reliability and real-time monitoring capability.
3. To develop a user-friendly web-based interface with Explainable AI visualization for real-time spoilage prediction, confidence analysis, and food quality monitoring.

1.6 Brief Methodology of the project

The methodology adopted for the project involves dataset collection, preprocessing, deep learning model development, IoT integration, evaluation, and deployment. Initially, images of fruits and vegetables representing different spoilage stages are collected from publicly available datasets such as Kaggle and other open-source sources. The collected images are categorized into Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage classes based on visible spoilage indicators such as discoloration, mold formation, dark spots, and texture degradation. The dataset is then preprocessed through resizing, normalization, cleaning, and augmentation techniques including rotation, flipping, zooming, and brightness adjustment to improve model performance and generalization capability.

Deep learning models such as CNN, MobileNetV2, ResNet50, and EfficientNetB0 are trained using transfer learning and fine-tuning approaches for spoilage classification. The dataset is divided into training, validation, and testing sets for performance evaluation. IoT sensors including MQ-135 gas sensors and DHT11/DHT22 temperature-humidity sensors are integrated using Arduino Uno or ESP32 microcontrollers to monitor environmental conditions influencing food spoilage. The image-based prediction confidence and sensor-based spoilage risk score are combined using a weighted fusion approach to generate the final spoilage prediction.

The trained models are evaluated using performance metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC analysis. Explainable AI using Grad-CAM visualization is incorporated to highlight image regions influencing the prediction process. Finally, a simple Streamlit-based web interface is developed for real-time image upload, spoilage prediction, confidence visualization, and sensor monitoring.

1.7 Assumptions made / Constraints of the project

The food spoilage detection process is assumed to operate under controlled image acquisition and environmental monitoring conditions. Images collected from publicly available datasets are assumed to represent different spoilage stages accurately and consistently. The deep learning models are trained assuming sufficient dataset quality, balanced class distribution, and proper preprocessing. IoT sensor readings obtained from MQ-135 and DHT11/DHT22 sensors are assumed to provide reliable measurements of gas concentration, temperature, and humidity related to spoilage conditions.

The proposed system mainly focuses on fruits and vegetables selected for the dataset and may not generalize completely to all food categories. Environmental conditions such as lighting variations, camera quality, background noise, and image resolution may influence prediction accuracy. The system also assumes stable internet and hardware connectivity for real-time deployment and sensor integration. Due to limited project duration and computational resources, the dataset size, number of food categories, and model optimization scope are restricted. The project is also limited to selected spoilage stages and predefined environmental thresholds used for sensor fusion and prediction analysis.

Chapter 2

**THEORY AND FUNDAMENTALS
OF AI-BASED FOOD SPOILAGE
DETECTION**

CHAPTER 2

THEORY AND FUNDAMENTALS OF AI-BASED FOOD SPOILAGE DETECTION

Artificial Intelligence (AI)-based food spoilage detection combines concepts from food science, computer vision, deep learning, image processing, Internet of Things (IoT), and Explainable AI to develop automated food quality monitoring systems. Food spoilage occurs due to microbial growth, enzymatic activity, oxidation, and environmental conditions such as temperature and humidity. Detecting spoilage at an early stage is important for reducing food waste and improving food safety. Traditional methods mainly rely on manual inspection based on visual appearance, odor, and texture, which are subjective and inconsistent. Recent advancements in deep learning and IoT technologies have enabled automated systems capable of identifying spoilage indicators accurately using image analysis and environmental sensing.

Deep learning models such as Convolutional Neural Networks (CNNs) and transfer learning architectures are widely used for food image classification because of their ability to automatically learn complex image features. IoT sensors such as MQ-135 gas sensors and DHT11/DHT22 temperature-humidity sensors are additionally used to monitor environmental conditions associated with spoilage progression. Explainable AI techniques such as Grad-CAM improve transparency by highlighting image regions influencing model predictions. This chapter discusses the theoretical concepts related to food spoilage, image processing, deep learning models, transfer learning, IoT sensing, sensor fusion, and Explainable AI used in the present study.

2.1 Fundamentals of Food Spoilage

Food spoilage is a complex process involving biochemical, physical, and microbial changes that render food unfit for consumption. In fruits and vegetables, this process is primarily driven by their high water activity and respiration rates. Microbial spoilage is caused by bacteria, yeasts, and molds that colonize the surface and internal tissues, leading to decomposition. Enzymatic spoilage, such as enzymatic browning, occurs when polyphenol oxidase enzymes react with oxygen, causing discoloration.

The progression of spoilage can be categorized into four distinct stages:

1. **Fresh:** The food item retains its original color, texture, and nutritional value with no visible signs of degradation.
2. **Early Spoilage:** Subtle changes appear, such as slight softening or minor color shifts. Ethylene gas emission may start increasing at this stage.
3. **Moderate Spoilage:** Visible mold growth, dark spots, and significant texture degradation become apparent. The release of volatile organic compounds (VOCs) increases.
4. **Severe Spoilage:** Extensive decay, liquefaction, and strong unpleasant odors characterize this stage, making the item hazardous for consumption.

2.2 Image Processing and Deep Learning for Spoilage Detection

Deep Learning, specifically Convolutional Neural Networks (CNNs), has revolutionized computer vision by eliminating the need for manual feature engineering. A standard CNN architecture consists of multiple layers:

- **Convolutional Layers:** Use learnable filters to detect local patterns like edges, textures, and color gradients associated with mold or bruises.
- **Pooling Layers:** Reduce spatial dimensions (e.g., Max Pooling) to achieve translation invariance and decrease computational load.
- **Fully Connected Layers:** Map the extracted high-level features to the final four classification categories.

Transfer Learning is employed to leverage models pretrained on the ImageNet dataset. This approach is highly effective for food spoilage detection where labeled datasets may be limited.

- **MobileNetV2:** Utilizes depthwise separable convolutions to provide a lightweight yet powerful model suitable for real-time mobile deployment.
- **ResNet50:** Uses residual blocks (skip connections) to overcome the vanishing gradient problem, allowing for deeper architectures and better feature extraction.

- **EfficientNetB0**: Implements compound scaling to balance network depth, width, and resolution, optimizing both accuracy and efficiency.

2.3 IoT Sensors and Environmental Monitoring

Environmental factors are critical indicators of the storage health of food products. The integration of IoT sensors provides a continuous data stream that complements visual analysis.

- **MQ-135 Gas Sensor**: This semiconductor sensor has a SnO₂ sensitive layer whose conductivity changes in the presence of gases like Ammonia (NH₃), Ethanol, and Carbon Dioxide (CO₂). These gases are typical byproducts of microbial respiration and fermentation in spoiling food.
- **DHT11/DHT22 Sensors**: These sensors measure ambient temperature and relative humidity. High temperature and humidity levels accelerate the growth of pathogens and spoilage organisms, significantly reducing the shelf life of produce.

The data from these sensors is transmitted to a microcontroller (ESP32 or Arduino Uno) where it is sampled and analyzed against predefined thresholds to calculate a real-time 'Environmental Risk Score'.

2.4 Explainable AI and Sensor Fusion

Explainable AI (XAI) addresses the 'black-box' nature of deep learning. **Grad-CAM** (Gradient-weighted Class Activation Mapping) uses the gradients of any target concept (e.g., 'Severe Spoilage') flowing into the final convolutional layer to produce a localization map. This heatmap highlights the specific pixels that influenced the model's decision, such as a patch of white mold on an orange or a dark bruise on a banana, thereby providing visual justification for the prediction.

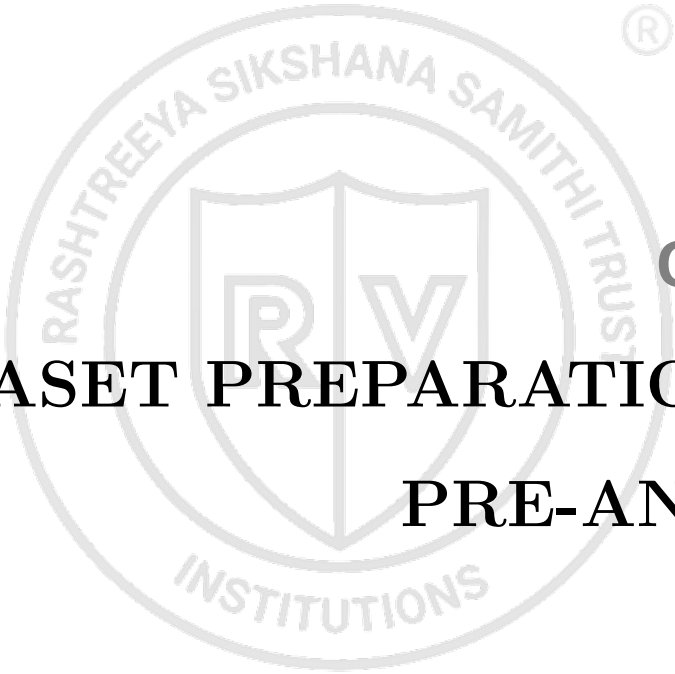
Sensor Fusion is implemented using a weighted decision mechanism. Let C_{img} be the confidence score from the deep learning model and R_{env} be the risk score derived from the IoT sensors. The final integrated score S_{final} is calculated as:

$$S_{final} = (\alpha \cdot C_{img}) + (\beta \cdot R_{env}) \quad (2.1)$$

where α and β are weights assigned based on the reliability of the image and sensor data

in a given environment. This fusion ensures that the system remains robust even if the visual data is obscured by poor lighting or if the sensor data is noisy.



The logo is a circular emblem. The outer ring contains the text "RASHTREEYA SIKSHANA SAMITHI TRUST" at the top and "INSTITUTIONS" at the bottom. In the center is a shield divided vertically, with a stylized "R" on the left and a "V" on the right. A registered trademark symbol (®) is located to the upper right of the logo.

Chapter 3

DATASET PREPARATION AND

PRE-ANALYSIS

CHAPTER 3

DATASET PREPARATION AND PRE-ANALYSIS

The effectiveness of deep learning-based food spoilage detection systems depends significantly on the quality of the dataset, pre-processing methods, labelling strategy, and training workflow. Proper dataset preparation improves classification performance, reduces overfitting, and enhances the generalization capability of deep learning models. Since the proposed system performs multi-label spoilage classification, careful organization and preprocessing of food images are necessary to ensure reliable prediction across different spoilage conditions. This chapter discusses dataset collection, spoilage stage labeling, preprocessing techniques, augmentation methods, dataset balancing, and the tools and hardware used for implementation.

3.1 Dataset Collection and Organization

The dataset for this project was curated by aggregating images from multiple high-quality sources, including the 'Fruits 360' dataset on Kaggle and various open-source computer vision repositories. To ensure the model is robust across different produce types, the collection focuses on three primary categories: tomatoes, bananas, and apples, which exhibit distinct visual spoilage patterns (e.g., softening and browning in bananas vs. mold and skin shriveling in tomatoes).

The total dataset comprises approximately 4,500 high-resolution images. These are organized into a hierarchical structure where the root directory contains sub-folders for each food type, which are further divided into the four target classes: *Fresh*, *Early Spoilage*, *Moderate Spoilage*, and *Severe Spoilage*. To maintain real-world applicability, the images include various backgrounds, occlusions, and lighting environments, ranging from natural sunlight to indoor artificial lighting.

3.2 Spoilage Stage Labelling Strategy

Since standard datasets often only provide binary labels, a rigorous labeling strategy was implemented. Each image was manually reviewed and cross-referenced with heuristic metrics to ensure objective classification:

- **Color Distribution Analysis:** Mean RGB and HSV values were calculated to detect shifts from vibrant colors (Fresh) to darker or brownish tones (Spoilage).

- **Darkness Ratio:** A thresholding algorithm was used to calculate the percentage of dark pixels, which often correlates with bruises or necrotic spots.
- **Texture Roughness:** Laplacian variance was used to measure image sharpness and surface texture; a decrease in variance often indicates the loss of surface integrity and softening.

This multi-modal labeling approach ensures that the ground truth for training is highly reliable, allowing the model to distinguish subtle differences between 'Early' and 'Moderate' spoilage.

3.3 Image Pre-processing Techniques

To standardize the input for deep learning models, several preprocessing steps are executed:

1. **Aspect-Aware Resizing:** Images are resized to 224×224 pixels using inter-linear interpolation. This specific resolution is the standard input size for the MobileNet and ResNet architectures, ensuring optimal feature map extraction.
2. **Channel Normalization:** Pixel values are scaled from $[0, 255]$ to $[0, 1]$ by dividing by 255.0. This prevents gradients from exploding and ensures faster convergence of the stochastic gradient descent (SGD) optimizer.
3. **Noise Reduction:** A 3×3 Gaussian blur kernel is applied to remove high-frequency noise that could otherwise lead the model to focus on irrelevant background details.

3.4 Data Augmentation Methods

To prevent overfitting and improve the generalization of the model to unseen household environments, a dynamic augmentation pipeline was implemented using the *ImageDataGenerator* class. The following transformations are applied randomly during each training epoch:

- **Rotation:** Random rotations between -20° and $+20^\circ$ to simulate different camera angles.
- **Flipping:** Horizontal and vertical flips to account for orientation variance.

- **Zoom and Shear:** Random zooming (range: 0.9 to 1.1) and shearing (range: 0.2) to simulate varying distances and perspective distortions.
- **Brightness adjustment:** Scaling pixel intensity by a factor of 0.8 to 1.2 to account for diverse lighting conditions.

3.5 Dataset Splitting and Balancing

The dataset is split using a stratified approach to maintain the class distribution across all subsets:

- **Training Set (70%):** Approximately 3,150 images used for updating weights.
- **Validation Set (15%):** 675 images used for monitoring the loss curve and early stopping.
- **Testing Set (15%):** 675 images reserved for the final unbiased evaluation of the system.

To address any minor class imbalances, *Synthetic Minority Over-sampling Technique* (SMOTE) or class weighting was applied in the loss function, ensuring the model does not become biased toward the 'Fresh' or 'Severe' categories which often have more samples.

3.6 Tools, Frameworks, and Software Used

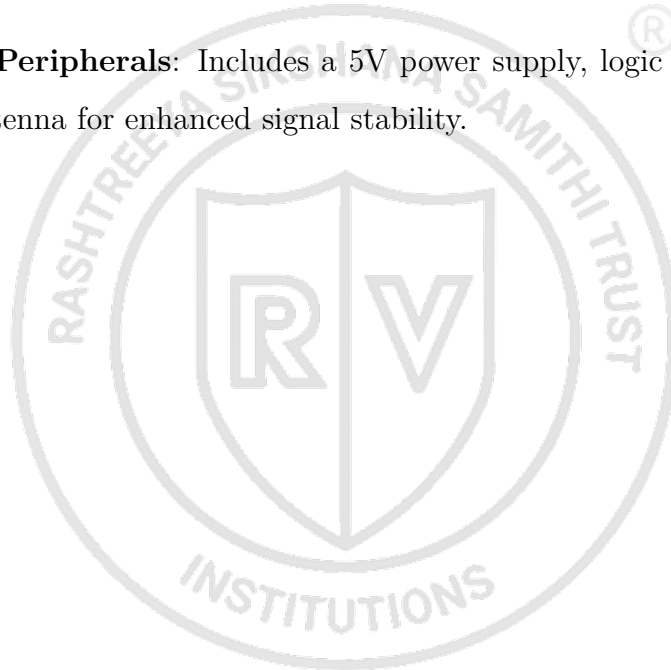
The software stack is built entirely on Python 3.9:

- **TensorFlow 2.x / Keras:** For building the multi-label classification models using the Functional API for complex architectures.
- **OpenCV:** For all low-level image processing and real-time camera interfacing.
- **Streamlit:** For the final deployment, providing a responsive UI for image uploads and real-time visualization of Grad-CAM heatmaps.
- **Matplotlib & Plotly:** For generating interactive training history logs and evaluation metrics like Confusion Matrices.

3.7 Hardware and IoT Components Used

The physical component of the system integrates several key hardware pieces:

- **MQ-135 Gas Sensor:** Capable of detecting Ammonia, Ethanol, and Smoke. It is calibrated by calculating the R_0 resistance in clean air to ensure accurate VOC detection.
- **DHT22 Sensor:** Chosen over the DHT11 for its higher precision ($\pm 0.5^\circ C$ for temperature and $\pm 2\%$ for humidity) and wider sensing range.
- **ESP32 Microcontroller:** A dual-core 240MHz processor with integrated WiFi/BLE, allowing for future expansion into cloud-based monitoring.
- **Standard Peripherals:** Includes a 5V power supply, logic level shifters, and an external antenna for enhanced signal stability.





Chapter 4

METHODOLOGY AND IMPLEMENTATION WORKFLOW

CHAPTER 4

METHODOLOGY AND IMPLEMENTATION

WORKFLOW

The proposed system combines deep learning, computer vision, IoT sensing, and Explainable AI techniques for automated food spoilage detection and monitoring. The methodology involves dataset collection, preprocessing, augmentation, model training, IoT integration, prediction fusion, Explainable AI visualization, and web-based deployment. The complete workflow is designed to perform multi-label spoilage classification of fruits and vegetables while incorporating environmental sensing for improved prediction reliability. The chapter discusses the overall system workflow, implementation methodology, and technologies used in the proposed system.

4.1 Overall Methodology

The methodology is structured as an end-to-end pipeline that synchronizes visual data with environmental sensor telemetry. The process begins with automated data ingestion from a local camera module and the IoT sensor array. Images are streamed as NumPy arrays, while sensor data (temperature, humidity, and VOC levels) are transmitted via Serial communication from the ESP32 to the host machine. This dual-stream approach ensures that the system has access to both the 'symptoms' of spoilage (visual indicators) and the 'causes' (environmental conditions).

The integration of these disparate data sources is managed by a centralized Python-based processing engine. The engine coordinates the execution of the deep learning inference and the sensor risk calculation in parallel, minimizing latency. This architecture supports real-time monitoring, where the prediction is updated dynamically as the environment changes, making it suitable for active storage tracking in smart kitchens and warehouses.

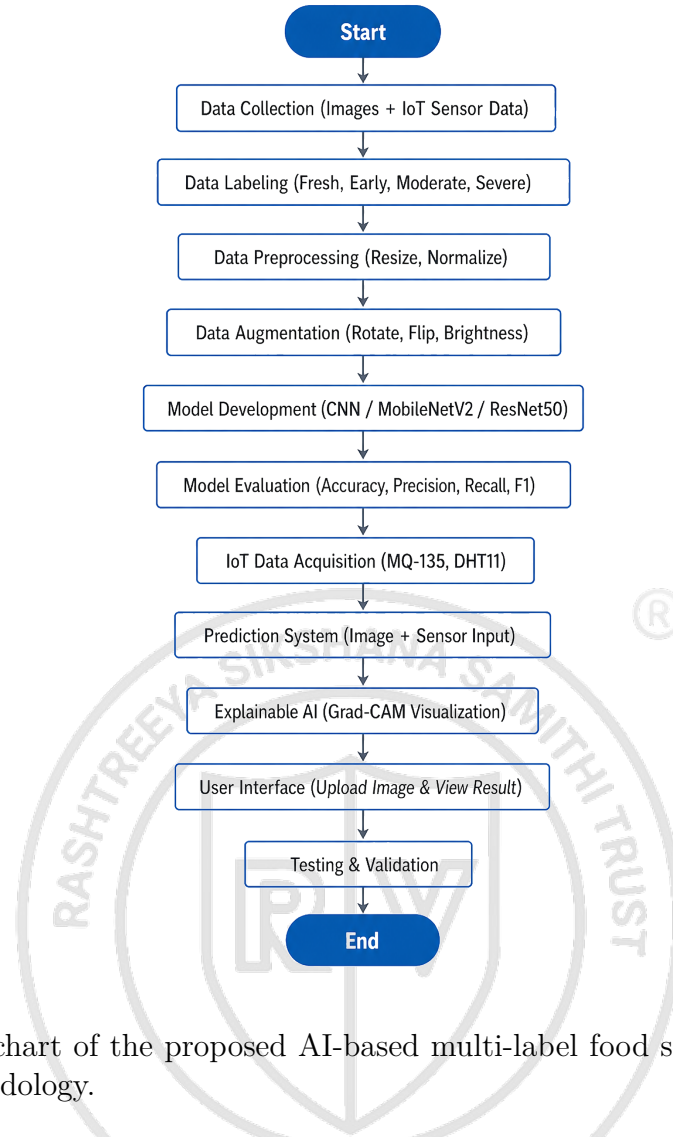


Figure 4.1: Flowchart of the proposed AI-based multi-label food spoilage detection and IoT fusion methodology.

Table 4.1: Methodology Workflow of the Proposed System

Stage	Description
Data Collection	Collection of fruit and vegetable images from public datasets
Data Labeling	Classification into Fresh, Early, Moderate, and Severe spoilage stages
Preprocessing	Image resizing, normalization, and cleaning
Data Augmentation	Rotation, flipping, zooming, and brightness adjustment
Model Training	Training CNN and transfer learning models
IoT Integration	Collection of environmental data using sensors
Sensor Fusion	Combination of image prediction and sensor risk score
Explainable AI	Grad-CAM visualization for prediction interpretation
Deployment	Streamlit-based web application development

4.2 Deep Learning Workflow

The deep learning training pipeline is optimized for high accuracy on multi-label classification. The models are implemented using the *Keras* API with the following

training configurations:

- **Loss Function:** Categorical Cross-Entropy, as it is a multi-class classification problem.
- **Optimizer:** *Adam* optimizer, starting with a learning rate of 0.001.
- **Fine-tuning:** After initial training of the top layers, the backbone layers (e.g., in MobileNetV2) are unfrozen and trained with a reduced learning rate of 10^{-5} to refine the lower-level features without destroying the pretrained weights.
- **Regularization:** Dropout layers (rate: 0.5) and L2 weight regularization are added to prevent the model from memorizing the training samples.

The training is performed with a batch size of 32 for 50 epochs, utilizing an *EarlyStopping* callback to halt training once the validation loss plateaus for more than 5 consecutive epochs.

4.3 IoT Data Acquisition and Sensor Integration

The IoT subsystem is designed for low-power, high-accuracy sensing. The microcontroller (ESP8266 or ESP32) samples data from the sensor array at a frequency of 1 Hz (2-second measurement intervals). To eliminate high-frequency electronic noise and jitters in the MQ-135 and DHT22 readings, a simple moving average filter with a window size of 5 is implemented on the firmware side.

The risk estimation logic is defined by a rule-based engine:

- **Thresholds:** Spoilage risk is flagged if Temperature exceeds 30°C , Humidity exceeds 80%, or VOC levels (from MQ-135) cross 200-400 ppm.
- **Risk Scoring:** These individual factors are combined into a normalized $[0, 1]$ risk score. This score represents the 'Environmental Spoilage Probability'.

The firmware integrates with the Blynk IoT cloud platform for real-time sensor monitoring and alert triggering. Virtual pins on Blynk display live gas concentration, temperature, and humidity readings. When spoilage conditions are detected, the system triggers cloud events that send push notifications and email alerts to the user. A local 16x2 I2C LCD display and tri-color LED indicator provide immediate visual feedback at

the device level. The complete ESP8266/Arduino firmware implementation is provided in Appendix Section A.8.

The sensor data is also formatted into a JSON string and sent to the Streamlit application via a virtual COM port or MQTT protocol, where it is used to adjust the final prediction confidence in the fusion module.

Table 4.2: IoT Sensors Used in the Project

Sensor	Parameter Monitored	Purpose
MQ-135	Gas concentration	Detection of spoilage-related gases
DHT11/DHT22	Temperature and Humidity	Monitoring environmental conditions
ESP32 / Arduino Uno	Sensor interfacing	Data acquisition and transmission

4.4 Explainable AI and Prediction System

The final prediction system employs a *Late Fusion* strategy. The image classification model outputs a probability distribution P_{DL} across the four classes, and the IoT risk score R_{env} provides a weight adjustment factor.

$$P_{final} = \text{Softmax}(P_{DL} + \lambda \cdot R_{env}) \quad (4.1)$$

where λ is a hyperparameter determined through validation to balance the influence of sensor data.

To provide user-centric explainability, **Grad-CAM** generates a saliency map of the input image. This map is calculated by taking the gradient of the winning class with respect to the feature maps of the last convolutional layer. The resulting heatmap is upsampled to 224×224 and superimposed on the original image using a 40% transparency alpha-blend. This allows the user to see exactly where the mold or discoloration was detected, transforming the system from a 'black box' into a transparent diagnostic tool. Finally, the Streamlit UI displays the real-time sensor charts, the prediction gauge, and the Grad-CAM output in a unified dashboard.

4.5 Detailed Hardware Configuration

This section provides comprehensive hardware implementation details including component specifications, wiring diagrams, sensor calibration thresholds, and cloud platform configuration for the IoT subsystem.

4.5.1 Hardware Components and Specifications

The IoT monitoring subsystem employs the following components:

- **Microcontroller:** NodeMCU ESP8266 with built-in Wi-Fi capability for cloud connectivity and real-time data transmission
- **Gas Sensor:** MQ-3/MQ-135 for detecting alcohol vapors, ammonia, and volatile organic compounds (VOCs) emitted during food decomposition with analog output range 0-1023
- **Environmental Sensor:** DHT11 for temperature and humidity monitoring with calibrated digital output
- **Display:** 16x2 I2C LCD display for local real-time sensor readings with automatic brightness control
- **Indicators:** Active piezo buzzer for acoustic alerts and dual-color LED (Green/Red) for visual status indication
- **Power:** 5V USB power supply for stable microcontroller and sensor operation

4.5.2 Comprehensive Wiring and Connection Reference

The following table provides a detailed pin-mapping reference for all hardware connections to the NodeMCU ESP8266 microcontroller:

Table 4.3: Complete Hardware Wiring Reference for NodeMCU ESP8266

Component	Component Pin	NodeMCU Pin	Description
I2C LCD Display	GND	GND	Ground connection
	VCC	VIN (5V)	Main power supply
	SDA	D2	I2C serial data line
	SCL	D1	I2C serial clock line
MQ-3/MQ-135	VCC	VIN (5V)	Power for internal heater element
	GND	GND	Ground connection
	A0 (Output)	A0	Analog signal output (0-1023)
DHT11 Sensor	VCC/+	3V3	3.3V logic power supply
	GND/-	GND	Ground connection
	DATA/OUT	D3	Digital data signal line
Active Buzzer	Positive (+)	D5	Digital control pin
	Negative (-)	GND	Ground connection
Green LED	Anode (Long)	D6	Digital control pin (via 330Ω resistor)
	Cathode (Short)	GND	Ground connection
Red LED	Anode (Long)	D7	Digital control pin (via 330Ω resistor)
	Cathode (Short)	GND	Ground connection

Note: All digital pins (D1-D7) operate at 3.3V logic level. LED connections require 330Ω current-limiting resistors to prevent damage to the microcontroller pins.

4.5.3 Sensor Calibration and Predictive Logic Thresholds

The gas sensor output is calibrated through experimental measurements to classify food spoilage into four distinct stages. The following thresholds are applied to the analog signal range (0-1023):

Table 4.4: Sensor Thresholds and Spoilage Stage Classification

Sensor Range	Spoilage Stage	LED Status	Buzzer	Cloud Alert
0 – 300	FRESH	Green ON / Red OFF	Silent	None
301 – 500	EARLY	Green ON / Red OFF	Silent	None
501 – 750	MODERATE	Green OFF / Red ON	Active	None
751 – 1023	SEVERE	Green OFF / Red ON	Active	Push + Email

These thresholds are determined empirically based on volatile organic compound (VOC) concentrations measured during controlled food decomposition experiments. The Fresh and Early stages operate silently to avoid false alarms during normal storage conditions. The Moderate stage activates local alerts (buzzer and red LED), while the Severe stage triggers cloud notifications (push notifications and email alerts) through the Blynk platform for immediate user notification.

4.5.4 Blynk Cloud Configuration and Automation

The Blynk IoT cloud platform enables real-time remote monitoring and automated alert delivery. The following configuration steps are required for complete system functionality:

Step 1: Template and Authentication Setup

- **Template ID:** TMPL3K_iKoy77
- **Template Name:** Food Spoilage Detection
- **Authentication Token:** Provided during Blynk device registration
- **Virtual Pins Configuration:**
 - **V2:** Gas Concentration (Gauge/Chart Widget for real-time and historical visualization)

- **V3**: Temperature (Value Widget displaying Celsius)
- **V4**: Humidity (Value Widget displaying relative humidity percentage)

Step 2: Event and Notification Setup

To enable push notifications and email alerts:

1. Log into **Blynk.Console** and navigate to **Developer Zone > Templates**
2. Open the food spoilage detection template
3. Click on the **Events & Notifications** tab and select **Edit** (top right corner)
4. Select or create the event code: `spoilage_alert`
5. Enable notifications by:
 - Toggling **Enable notifications** switch to ON
 - Setting **Email To**: Device Owner
 - Setting **Push Notifications To**: Device Owner
6. Click **Save** and then the green **Save And Apply** button at the top right

Step 3: Mobile App Configuration

- Install the Blynk mobile application (iOS or Android)
- Add a new device using the authentication token
- Add virtual pin widgets to the mobile interface:
 - Gauge widget for real-time gas concentration monitoring (V2)
 - Value widgets for temperature (V3) and humidity (V4)
 - Chart widgets for historical trend visualization and pattern analysis
- Enable push notifications in app settings
- Configure email notification delivery preferences for alert recipients

Alert Triggering Logic

When the gas sensor value exceeds 750 ppm (Severe Spoilage threshold), the firmware executes the alert triggering mechanism:

Listing 4.1: Blynk alert triggering mechanism for severe spoilage

```
if (sensorValue > 750) {  
    stageText = "SEVERE";  
    digitalWrite(BUZZER_PIN, HIGH);  
    digitalWrite(RED_LED, HIGH);  
  
    if (!alertTriggered) {  
        Blynk.logEvent("spoilage_alert");  
        alertTriggered = true;  
    }  
}
```

This ensures that alerts are fired only once when severe spoilage is detected, preventing notification spam while maintaining the alert state in the cloud platform for historical logging and analysis.

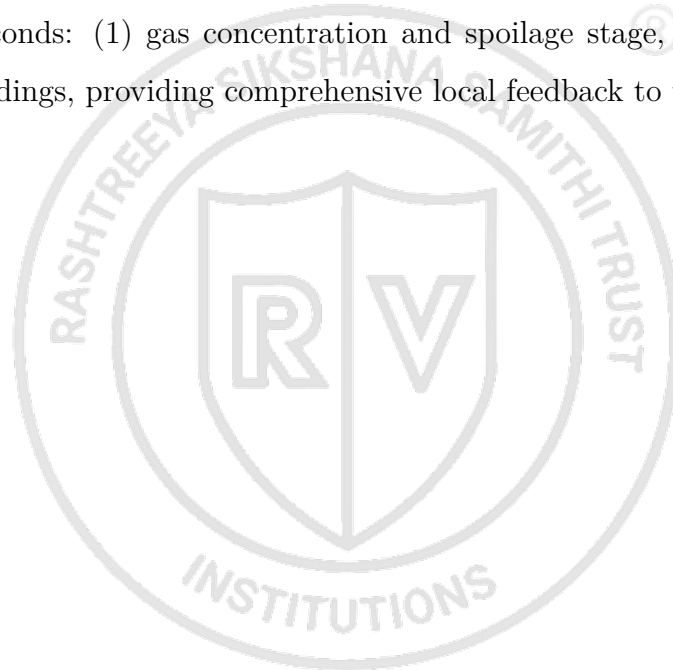
4.5.5 Power Management and System Initialization Sequence

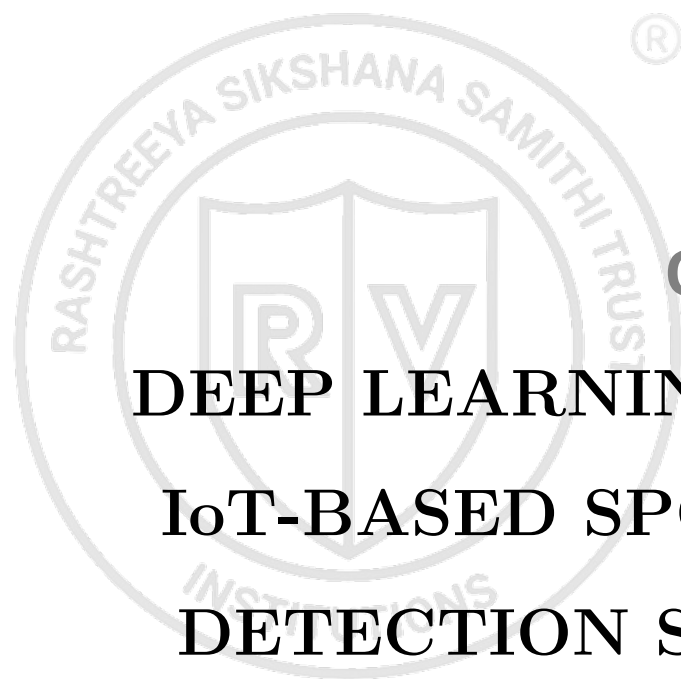
The system initializes in a carefully orchestrated sequence to ensure safe operation and reliable sensor readings:

1. **Power On:** All GPIO pins (D1-D7) initialized to LOW (safe state) to prevent accidental activation
2. **LCD Display:** Shows "System Booting..." and "Waking Sensors..." during initialization
3. **Sensor Warm-up:** DHT sensor initialization with internal warm-up delay for stable readings
4. **Cloud Connection:** Blynk cloud connection establishment via mobile hotspot Wi-Fi

5. **Timer Configuration:** Sensor monitoring interval set to 2000 milliseconds (2-second cycle)
6. **Status Confirmation:** LCD displays "System Online!" confirming successful initialization
7. **Operational Mode:** System enters continuous monitoring loop with 2-second measurement intervals

The 2-second measurement interval balances real-time responsiveness with sensor stability, ensuring accurate readings without overwhelming the microcontroller or cloud platform with excessive data transmission. The LCD display alternates between two views every 2 seconds: (1) gas concentration and spoilage stage, and (2) temperature and humidity readings, providing comprehensive local feedback to the user.





Chapter 5

DEEP LEARNING AND IoT-BASED SPOILAGE DETECTION SYSTEM

CHAPTER 5

DEEP LEARNING AND IOT-BASED SPOILAGE DETECTION SYSTEM

The proposed system integrates Artificial Intelligence (AI), Deep Learning, Computer Vision, Internet of Things (IoT), and Explainable AI techniques to develop an automated food spoilage detection and monitoring system. The system is designed to classify fruits and vegetables into four spoilage stages: Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage. The complete workflow consists of dataset preparation, image preprocessing, data augmentation, deep learning model training, IoT sensor integration, prediction fusion, Explainable AI visualization, and deployment through a Streamlit-based web application. The integration of image analysis and environmental sensing improves prediction reliability and enables real-time food quality monitoring. This chapter discusses the architecture, implementation details, and working principles of the proposed system.

5.1 System Architecture

The overall architecture of the proposed system consists of multiple interconnected modules working together for food spoilage detection and prediction. The system mainly contains the image-based deep learning module, IoT-based environmental monitoring module, prediction fusion module, Explainable AI module, and web application interface. The image analysis component processes food images and extracts visual features associated with spoilage, while the IoT sensing component monitors environmental parameters influencing spoilage progression.

Initially, food images are collected from publicly available datasets and organized according to spoilage stages. These images are passed through preprocessing and augmentation pipelines before being used for model training. The deep learning module performs spoilage classification using CNN and transfer learning architectures. Simultaneously, environmental sensor data are collected using MQ-135 gas sensors and DHT11/DHT22 temperature-humidity sensors connected through Arduino Uno or ESP32 microcontrollers.

The prediction outputs from the image analysis module and IoT sensor module are combined using a weighted fusion mechanism to generate the final spoilage stage prediction. Explainable AI techniques such as Grad-CAM generate heatmaps highlighting

important image regions influencing predictions. The final output is displayed through a Streamlit-based web application where users can upload images, monitor sensor readings, and visualize spoilage predictions in real time.

Table 5.1: Major Components of the Proposed System

Component	Function
Dataset Module	Collection and organization of food images
Preprocessing Module	Image resizing, normalization, and augmentation
Deep Learning Module	Spoilage stage classification
IoT Sensor Module	Environmental monitoring
Fusion Module	Combination of image and sensor predictions
Explainable AI Module	Grad-CAM visualization
Web Interface	User interaction and prediction display

5.2 Deep Learning Model Development

Deep learning models are used for image-based spoilage classification because of their strong feature extraction and pattern recognition capabilities. Convolutional Neural Networks (CNNs) are highly effective in image classification tasks because they automatically learn hierarchical features such as edges, textures, patterns, and object structures from images. In food spoilage detection, CNN models learn visual spoilage indicators including mold growth, discoloration, bruises, dark spots, and texture degradation from fruit and vegetable images.

The proposed system uses CNN-based transfer learning architectures such as MobileNetV2, ResNet50, and EfficientNetB0. Transfer learning improves classification performance by utilizing pretrained ImageNet weights and fine-tuning the models for food spoilage detection. This reduces training complexity and improves prediction accuracy even when the available dataset size is limited. The classification head added to the pretrained backbone consists of fully connected dense layers, batch normalization layers, activation functions, and dropout layers for regularization.

The models are trained using categorical cross-entropy loss and Adam optimization algorithms. Batch normalization improves training stability, while dropout layers reduce overfitting by randomly disabling neurons during training. Training is performed in two stages. During the first stage, pretrained backbone layers are frozen and only the classification layers are trained. During the second stage, selected backbone layers are unfrozen and fine-tuned using a smaller learning rate to improve feature extraction ca-

pability. Performance evaluation is performed using accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC analysis.

Table 5.2: Deep Learning Models Used

Model	Purpose	Advantage
CNN	Baseline spoilage classification	Simple architecture
MobileNetV2	Lightweight transfer learning	Fast inference speed
ResNet50	Deep feature extraction	Improved classification accuracy
EfficientNetB0	Optimized transfer learning	Better efficiency and performance

5.3 IoT-Based Environmental Monitoring

Environmental conditions significantly influence food spoilage and microbial growth. Temperature, humidity, and gas concentration affect the rate of decomposition, microbial activity, and spoilage progression in fruits and vegetables. Therefore, IoT-based environmental monitoring is integrated into the proposed system to improve prediction reliability and support real-time spoilage analysis.

The system uses MQ-135 gas sensors to detect gases and volatile compounds released during food decomposition. The sensor is sensitive to gases such as ammonia, benzene, smoke, and volatile organic compounds associated with spoilage. DHT11/DHT22 sensors are used to monitor environmental temperature and humidity conditions. These sensors are connected to microcontrollers such as Arduino Uno or ESP32 microcontrollers for continuous data acquisition and communication.

The collected sensor readings are processed using threshold-based analysis to estimate spoilage risk. High temperature, humidity, and gas concentration values increase the environmental spoilage risk score. The IoT module continuously monitors environmental conditions and provides additional information beyond image-based prediction. This enables early spoilage detection even before severe visual deterioration becomes visible.

Table 5.3: IoT Hardware Components

Hardware Component	Function
MQ-135 Gas Sensor	Detection of spoilage-related gases
DHT11/DHT22 Sensor	Temperature and humidity monitoring
Arduino Uno / ESP32	Sensor interfacing and data acquisition
Breadboard and Jumper Wires	Circuit connections
Power Supply	Hardware operation

5.4 Sensor Fusion and Prediction System

The proposed system combines image-based deep learning predictions and IoT sensor-based spoilage analysis using a weighted fusion mechanism. The deep learning module generates probability scores for each spoilage stage based on visual image analysis, while the IoT module generates environmental spoilage risk scores based on sensor readings. These outputs are combined mathematically to generate the final spoilage stage prediction.

The weighted fusion approach improves system robustness because both visual and environmental information contribute to prediction generation. In some cases, visual spoilage symptoms may not be clearly visible, but unfavorable environmental conditions such as high humidity or gas concentration may indicate early spoilage progression. The sensor fusion mechanism enables the system to provide more reliable and practical predictions compared to conventional image-only approaches.

The final prediction output includes the predicted spoilage stage, confidence score, Grad-CAM visualization, and IoT sensor status. The prediction pipeline supports real-time food quality monitoring and enables practical deployment for household kitchens, grocery stores, warehouses, and smart inventory management systems.

Table 5.4: Sensor Fusion Parameters

Input Source	Weight Contribution
Image-Based Prediction	0.75
IoT Sensor Risk Score	0.25

5.5 Explainable AI Using Grad-CAM

Explainable Artificial Intelligence (XAI) techniques are integrated into the proposed system to improve transparency and interpretability of deep learning predictions. Most deep learning systems operate as black-box models and do not explain how predictions are generated. In practical food monitoring systems, understanding the prediction process is important for improving user trust and validating model behaviour.

The proposed system uses Grad-CAM (Gradient-weighted Class Activation Mapping) to generate heatmaps highlighting image regions influencing the classification result. Grad-CAM uses gradient information from the final convolutional layers to identify important regions contributing to the prediction. The generated heatmaps highlight

spoilage indicators such as mold patches, bruises, discoloration, dark spots, and texture degradation.

The Explainable AI module improves transparency by allowing users to understand whether the model focuses on meaningful spoilage regions during prediction. This also helps validate model performance and improves reliability of the AI-based spoilage detection system for practical real-world applications.

5.6 Streamlit-Based Web Interface

A simple and interactive web interface is developed using Streamlit for real-time spoilage prediction and monitoring. The web application allows users to upload images of fruits and vegetables and instantly view spoilage predictions generated by the trained deep learning model. The interface displays the predicted spoilage stage, confidence score, Grad-CAM visualization, and IoT sensor readings.

The Streamlit-based interface follows a clean and minimal design inspired by food freshness and usability principles. Users can interact with the system through image upload and drag-and-drop functionality. The web application also supports visualization of environmental sensor data and spoilage risk analysis. The deployment demonstrates the practical implementation of the proposed AI-IoT spoilage detection system for food quality monitoring and food waste reduction applications.



Chapter 6

RESULTS AND DISCUSSION

CHAPTER 6

RESULTS AND DISCUSSION

The performance of the proposed AI-based multi-label food spoilage detection system is evaluated using deep learning performance metrics, visualization techniques, IoT sensor analysis, and Explainable AI outputs. The trained models are tested on unseen images to evaluate their ability to classify fruits and vegetables into Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage stages. The evaluation process includes analysis of training and validation performance, confusion matrix interpretation, ROC analysis, Grad-CAM visualization, and IoT sensor monitoring results. This chapter discusses the experimental results obtained from the proposed system and analyzes the effectiveness of the implemented methodologies.

6.1 Dataset Distribution and Visualization

The dataset used for training and evaluation consists of fruit and vegetable images categorized into four spoilage stages. The dataset is divided into training, validation, and testing sets using a standard split ratio to ensure effective model training and unbiased evaluation. Data augmentation techniques such as rotation, flipping, zooming, and brightness adjustment are applied to improve dataset diversity and model robustness.

Visualization of the dataset distribution helps analyze class balance and spoilage stage representation. Sample image grids are generated for all spoilage stages to verify dataset quality and labeling consistency. Pixel intensity distribution analysis is also performed to understand visual variations between spoilage stages.

Table 6.1: Dataset Distribution

Dataset	Split Percentage
Training Set	70%
Validation Set	15%
Testing Set	15%

6.2 Model Training and Validation Performance

The proposed food spoilage detection system was trained and evaluated using four deep learning architectures: CustomCNN, MobileNetV2, ResNet50, and EfficientNetB0. All models were trained using the same 70:15:15 dataset split, identical augmentation techniques, class weights, and preprocessing pipeline to ensure fair comparison. Per-

formance evaluation was carried out using Accuracy, Precision, Recall, F1-Score, and ROC-AUC metrics.

The comparative analysis results indicate that MobileNetV2 achieved the best overall performance among all evaluated models. MobileNetV2 obtained an accuracy of 62.75%, precision of 72.05%, recall of 62.75%, F1-score of 65.47%, and ROC-AUC value of 85.74%. The model demonstrated better generalization capability and more stable classification performance across different spoilage stages compared to the other architectures. The lightweight architecture and depthwise separable convolution operations used in MobileNetV2 improved feature extraction efficiency while maintaining lower computational complexity.

The CustomCNN model achieved moderate performance with an accuracy of 34.07%, precision of 62.45%, recall of 34.07%, F1-score of 33.44%, and ROC-AUC value of 80.71%. Although the model showed lower classification accuracy compared to transfer learning architectures, it demonstrated good ROC performance and faster inference capability because of its smaller architecture size. The CustomCNN model served as a baseline architecture for comparison against pretrained transfer learning models.

ResNet50 achieved an accuracy of 51.47%, precision of 39.88%, recall of 51.47%, F1-score of 37.46%, and ROC-AUC value of 64.04%. The model demonstrated strong feature extraction capability for Fresh samples but showed difficulty in correctly distinguishing Early Spoilage and Severe Spoilage stages. The deeper architecture increased model complexity and training time without providing significant improvement in classification performance for the current dataset.

EfficientNetB0 also achieved an accuracy of 51.47% with a precision of 30.15%, recall of 51.47%, F1-score of 36.17%, and ROC-AUC value of 51.47%. Although EfficientNetB0 is known for parameter efficiency and optimized scaling, the model showed inconsistent classification performance for the present multi-label spoilage dataset. The results suggest that EfficientNetB0 may require larger datasets and further hyperparameter tuning for improved performance in spoilage classification tasks.

...

=====

FINAL MODEL COMPARISON TABLE

=====

	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Train Time (min)	Model Size (MB)
Model							
CustomCNN	0.3407	0.6245	0.3407	0.3344	0.8071	15.2	5.6
MobileNetV2	0.6275	0.7205	0.6275	0.6547	0.8574	12.7	31.3
ResNet50	0.5147	0.3988	0.5147	0.3746	0.6404	13.8	244.8
EfficientNetB0	0.5147	0.3015	0.5147	0.3617	0.5147	15.8	38.4

Best per metric:

Accuracy : MobileNetV2 (0.6275)

Precision : MobileNetV2 (0.7205)

Recall : MobileNetV2 (0.6275)

F1-Score : MobileNetV2 (0.6547)

ROC-AUC : MobileNetV2 (0.8574)

Figure 6.1: Final comparative performance analysis of deep learning models including training time and model size.

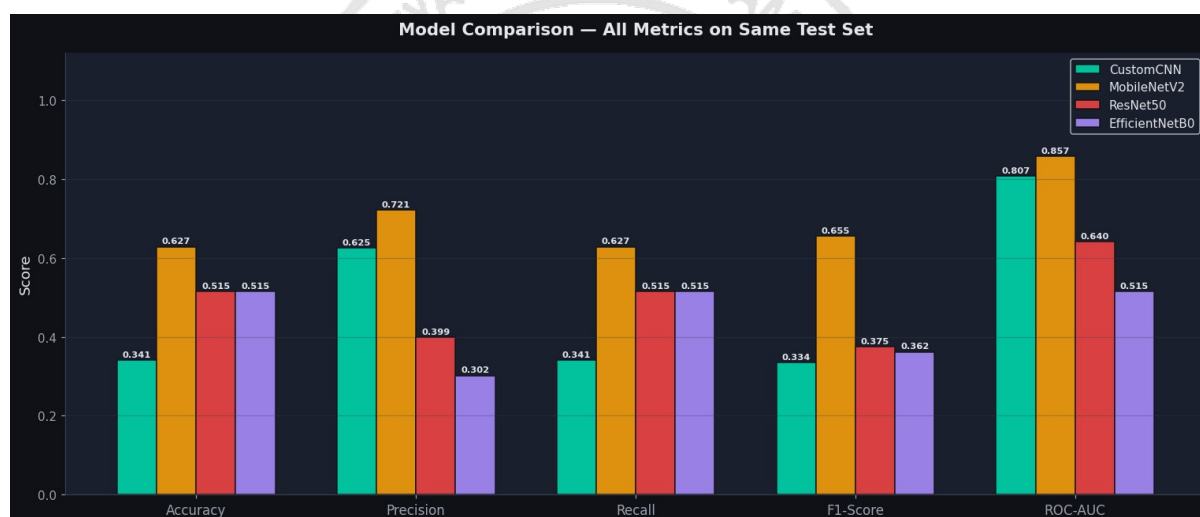


Figure 6.2: Model comparison across all metrics on the same test set.

Table 6.2: Comparative Performance Analysis of Deep Learning Models

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
CustomCNN	34.07%	62.45%	34.07%	33.44%	80.71%
MobileNetV2	62.75%	72.05%	62.75%	65.47%	85.74%
ResNet50	51.47%	39.88%	51.47%	37.46%	64.04%
EfficientNetB0	51.47%	30.15%	51.47%	36.17%	51.47%

6.3 Confusion Matrix and Classification Analysis

Confusion matrix analysis was performed to evaluate stage-wise classification capability of the trained models. The confusion matrices indicate that MobileNetV2 provided

the most balanced classification performance across all spoilage stages. The model correctly identified Fresh samples with high accuracy while also demonstrating improved prediction capability for Moderate and Severe Spoilage classes. Compared to the other architectures, MobileNetV2 produced fewer false classifications and achieved better class-wise balance.

The CustomCNN model showed moderate performance for Severe Spoilage classification but struggled to differentiate Early Spoilage and Moderate Spoilage stages accurately. ResNet50 and EfficientNetB0 strongly favored Fresh class predictions and misclassified many spoiled samples as Fresh, resulting in reduced F1-score and poor class-wise balance. This indicates that deeper architectures may require larger and more balanced datasets to effectively learn subtle spoilage variations between intermediate spoilage stages.

Per-class F1-score analysis further confirmed that MobileNetV2 achieved the best overall class-wise balance among all evaluated models. The model achieved higher F1-scores for Fresh, Moderate Spoilage, and Severe Spoilage classes compared to the remaining architectures. The results demonstrate that MobileNetV2 is the most suitable architecture for the current food spoilage detection dataset and implementation workflow.

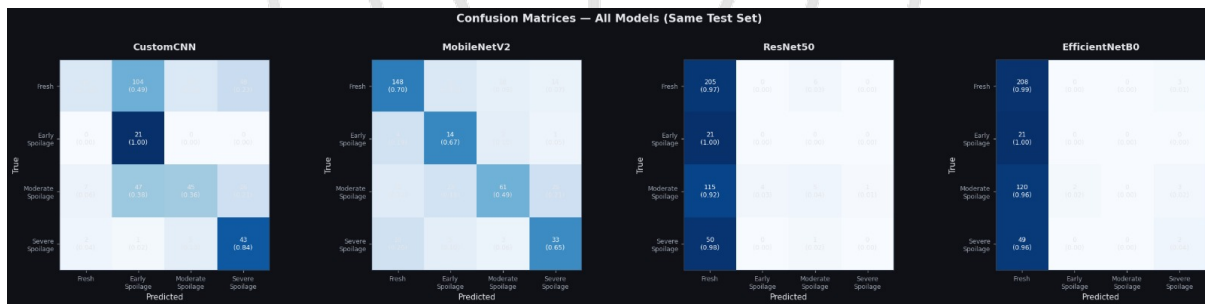


Figure 6.3: Confusion Matrices for CustomCNN, MobileNetV2, ResNet50, and EfficientNetB0 on the same test set.

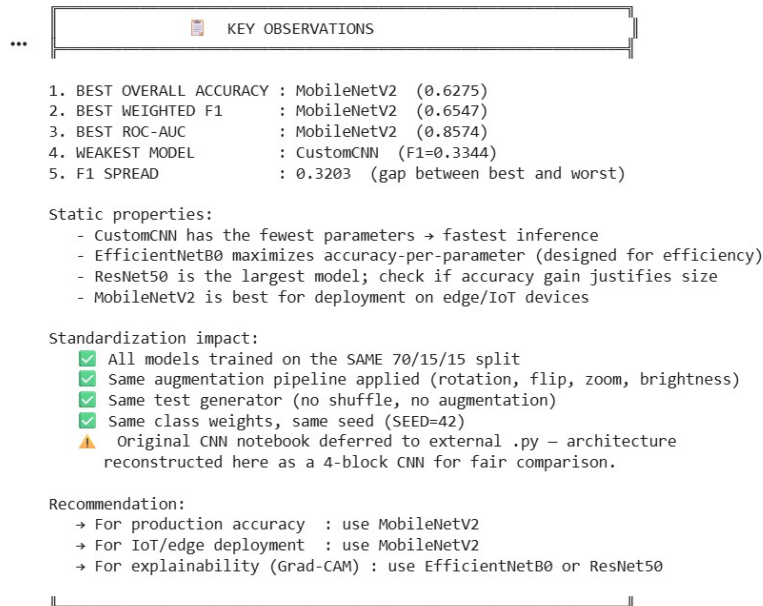


Figure 6.4: Key observations and summary of findings from the model evaluation.

Table 6.3: Key Observations from Comparative Analysis

Observation	Result
Best Overall Accuracy	MobileNetV2 (62.75%)
Best Weighted F1-Score	MobileNetV2 (65.47%)
Best ROC-AUC	MobileNetV2 (85.74%)
Fastest Lightweight Model	MobileNetV2
Weakest Overall Model	CustomCNN
Largest Model Size	ResNet50

6.4 ROC Curve and Per-Class Performance Analysis

ROC curve analysis was performed using a one-vs-rest classification strategy for all spoilage stages. The ROC curves indicate that MobileNetV2 achieved the highest Area Under Curve (AUC) values across most spoilage classes, demonstrating stronger class separability and prediction reliability. The model achieved particularly high ROC performance for Fresh and Severe Spoilage stages.

The CustomCNN model demonstrated acceptable ROC performance despite lower overall classification accuracy, indicating that the model was capable of learning important visual features but lacked strong class-wise generalization. ResNet50 and EfficientNetB0 produced weaker ROC curves for Early and Moderate Spoilage stages, suggesting difficulty in learning subtle spoilage transitions from the available dataset.

Per-class F1-score analysis revealed that Fresh samples were generally classified more accurately across all architectures because of their strong visual distinction. However, intermediate spoilage stages such as Early Spoilage and Moderate Spoilage were more difficult to classify due to overlapping visual characteristics and gradual spoilage progression. This highlights the complexity of multi-label food spoilage classification compared to conventional binary classification systems.

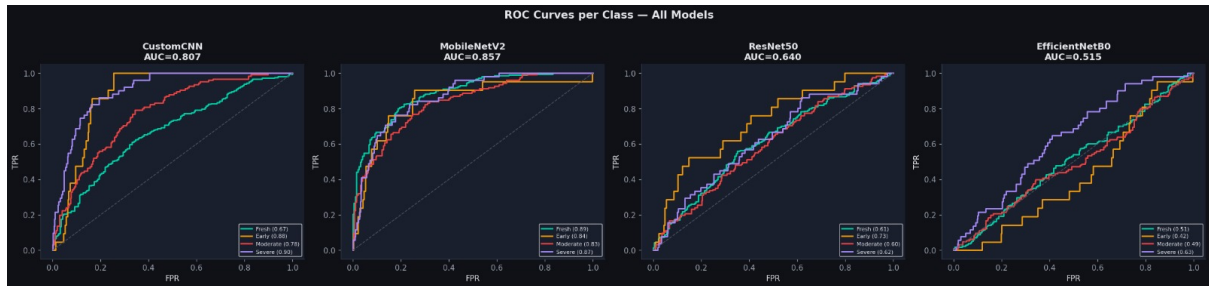


Figure 6.5: ROC Curves per class for all evaluated deep learning models.



Chapter 7

CONCLUSION AND FUTURE

SCOPE

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

The proposed AI-based multi-label food spoilage detection system demonstrates the application of Deep Learning, Computer Vision, and Explainable AI techniques for automated food quality monitoring. The system was developed to classify fruits and vegetables into four spoilage stages: Fresh, Early Spoilage, Moderate Spoilage, and Severe Spoilage using image-based analysis. Multiple deep learning architectures including CustomCNN, MobileNetV2, ResNet50, and EfficientNetB0 were trained and evaluated using the same dataset and pre-processing pipeline for comparative analysis.

The experimental results indicate that MobileNetV2 achieved the best overall performance among all evaluated models with improved accuracy, F1-score, and ROC-AUC values. The lightweight architecture and efficient feature extraction capability of MobileNetV2 enabled better classification performance and faster inference compared to deeper architectures such as ResNet50 and EfficientNetB0. The confusion matrix and ROC analysis demonstrated that the proposed system was capable of performing effective multi-label spoilage classification, although intermediate spoilage stages remained more challenging because of overlapping visual characteristics.

The integration of Explainable AI using Grad-CAM improved model transparency and interpretability by highlighting image regions influencing predictions. The generated heatmaps demonstrated that the models focused on meaningful spoilage indicators such as discoloration, mold formation, bruises, and texture degradation during classification. The Streamlit-based web application further demonstrated the practical applicability of the proposed system for real-time food quality monitoring and prediction.

Although the IoT integration component is currently under development, the proposed framework successfully establishes the foundation for combining image-based deep learning and environmental sensing for food spoilage monitoring. The project demonstrates the potential of AI-driven systems for reducing food waste, improving food safety, and enabling intelligent food quality assessment in practical applications such as smart kitchens, grocery stores, warehouses, and inventory management systems.

7.1 Future Scope

The future scope of the project includes complete integration of IoT-based environmental sensing using MQ-135 gas sensors and DHT11/DHT22 temperature-humidity sensors for real-time spoilage monitoring. Sensor fusion techniques can further improve prediction reliability by combining environmental conditions with image-based analysis. Additional food categories such as dairy products, packaged foods, and meat products can also be included to expand the applicability of the system.

Future work may also involve the use of larger and more diverse datasets to improve model generalization capability and classification accuracy for intermediate spoilage stages. Advanced architectures such as Vision Transformers (ViTs), multimodal learning models, and ensemble methods can be explored for improved performance. Cloud deployment, mobile application integration, and edge-AI implementation using embedded devices can further enhance the real-world usability of the proposed system.

7.2 Learning Outcomes of the Project

The project provided practical understanding of Deep Learning, transfer learning, image pre-processing, data augmentation, Explainable AI, and web application deployment techniques. The implementation process improved knowledge related to CNN architectures, transfer learning models, Grad-CAM visualization, and performance evaluation methods such as confusion matrix and ROC analysis.

The project also provided exposure to AI-based food quality monitoring systems and the integration of computer vision techniques with IoT-based monitoring frameworks. Practical implementation using TensorFlow, Keras, OpenCV, Streamlit, and Google Colab enhanced technical understanding of real-world AI application development workflows.

APPENDIX

The appendix presents selected implementation details, sample code snippets, and supporting outputs used during the development of the proposed AI-based multi-label food spoilage detection system. The implementation was carried out using Python in the Google Colab environment with TensorFlow, Keras, OpenCV, NumPy, Matplotlib, and Streamlit libraries. The appendix provides an overview of important code sections used for dataset preprocessing, model development, prediction generation, Explainable AI visualization, and web application deployment.

The developed system utilized transfer learning architectures such as MobileNetV2 and ResNet50 for spoilage classification. Image preprocessing operations including resizing, normalization, and augmentation were implemented to improve model training and generalization capability. Grad-CAM visualization techniques were also integrated to improve interpretability of model predictions. The following code snippets represent important implementation modules used in the project.

A.1 Importing Required Libraries

The following libraries were imported for image preprocessing, deep learning model development, visualization, and performance evaluation. TensorFlow and Keras were used for model training, while OpenCV and NumPy were used for image processing operations.

Listing 1: Importing Python libraries

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import MobileNetV2, ResNet50

import cv2
import numpy as np
import matplotlib.pyplot as plt
```

A.2 Dataset Preprocessing and Augmentation

Image preprocessing and augmentation techniques were applied to improve dataset quality and increase data diversity. Augmentation operations such as rotation, zooming, horizontal flipping, and brightness adjustment helped reduce overfitting and improve model generalization capability.

Listing 2: Dataset preprocessing and augmentation pipeline

```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=30,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    brightness_range=[0.7,1.3]  
)  
  
train_generator = train_datagen.flow_from_directory(  
    '/content/data/train',  
    target_size=(224,224),  
    batch_size=32,  
    class_mode='categorical'  
)
```

A.3 MobileNetV2 Model Architecture

Transfer learning was implemented using MobileNetV2 pretrained on the ImageNet dataset. The pretrained backbone was used for feature extraction, while additional dense and dropout layers were added for spoilage stage classification.

Listing 3: MobileNetV2 model construction

```
base_model = MobileNetV2(  
    include_top=False,  
    weights='imagenet',  
    input_shape=(224,224,3)  
)
```

```
base_model.trainable = False

model = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(4, activation='softmax')
])
```

A.4 Model Compilation and Training

The model was compiled using the Adam optimizer and categorical cross-entropy loss function. Training and validation datasets were used to evaluate model learning performance during multiple training epochs.

Listing 4: Model compilation and training process

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=20
)
```

A.5 Prediction Function

The prediction pipeline processed uploaded food images and generated spoilage stage predictions using the trained deep learning model. The predicted class was obtained using the highest probability score from the softmax output layer.

Listing 5: Prediction function for multi-label classification

```
def predict_image(image_path):  
    img = cv2.imread(image_path)  
    img = cv2.resize(img, (224,224))  
    img = img / 255.0  
    img = np.expand_dims(img, axis=0)  
  
    prediction = model.predict(img)  
    predicted_class = np.argmax(prediction)  
  
    return predicted_class
```

A.6 Grad-CAM Visualization

Grad-CAM was implemented to generate heatmaps highlighting important image regions influencing the prediction process. The Explainable AI component improved transparency and interpretability of the deep learning model.

Listing 6: Grad-CAM heatmap implementation

```
def gradcam_heatmap(img_array, model):  
    grad_model = tf.keras.models.Model(  
        [model.inputs],  
        [model.get_layer().output, model.output]  
    )
```

A.7 Streamlit Web Interface

A Streamlit-based web application was developed to provide a simple and interactive interface for real-time spoilage prediction. Users could upload fruit or vegetable images and obtain spoilage predictions instantly.

Listing 7: Streamlit web application snippet

```
import streamlit as st  
  
st.title("AI-Based Food Spoilage Detection")
```



```
uploaded_file = st.file_uploader(
    "Upload Fruit or Vegetable Image"
)

if uploaded_file is not None:
    st.image(uploaded_file)
```

A.8 Performance Evaluation Metrics

Different performance evaluation metrics such as Accuracy, Precision, Recall, and F1-Score were used to analyze the classification performance of the proposed models.

Listing 8: Evaluation metrics import

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

A.9 IoT Firmware: ESP8266 Blynk Integration

The IoT subsystem firmware was implemented on an ESP8266 microcontroller to interface with MQ-135 gas sensors and DHT11/DHT22 temperature-humidity sensors. The firmware integrates with Blynk cloud platform for real-time data transmission, local LCD display monitoring, and automated alert triggering. The following code represents the complete Arduino sketch for the IoT sensor integration module.

Listing 9: ESP8266 Blynk firmware for food spoilage monitoring

```
#define BLYNK_TEMPLATE_ID "TMPL3K_iKoy77"
#define BLYNK_TEMPLATE_NAME "Food Spoilage detections"
#define BLYNK_AUTH_TOKEN "YrmWQOMAA8Fs9_0OmlkreAO5kcXZ3Vx3"

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ESP8266WiFi.h>
```

```
#include <BlynkSimpleEsp8266.h>
#include <DHT.h>

// Mobile Hotspot Credentials
const char* ssid = "OnePlus_Nord_CE5_7DCA";
const char* password = "123456789";

// Pin Configurations
const int MQ3_PIN = A0;
#define DHTPIN D3
#define DHTTYPE DHT11
#define BUZZER_PIN D5
#define GREEN_LED D6
#define RED_LED D7

// Component Initializations
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHTPIN, DHTTYPE);
BlynkTimer timer;

// Control Flags
bool alertTriggered = false;
bool displayToggle = true;

void monitorSystem() {
    // Read Sensor Telemetry
    int sensorValue = analogRead(MQ3_PIN);
    float humidity = dht.readHumidity();
    float temperature = dht.readTemperature();

    // Handle Sensor Errors Gracefully
    if (isnan(humidity) || isnan(temperature)) {
```

```
    humidity = 0;
    temperature = 0;
}

// Push Stream Data to Blynk Virtual Pins
Blynk.virtualWrite(V2, sensorValue);    // Gauge / Chart Widget
Blynk.virtualWrite(V3, temperature);    // Temp Value Widget
Blynk.virtualWrite(V4, humidity);       // Humidity Value Widget

String stageText = "FRESH";

// 4-Stage State Machine Integration
if (sensorValue > 750) {
    stageText = "SEVERE";
    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);

    // Fire cloud automation event (Triggers Push and Email)
    if (!alertTriggered) {
        Blynk.logEvent("spoilage_alert");
        alertTriggered = true;
    }
}

else if (sensorValue > 500) {
    stageText = "MODERATE";
    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    alertTriggered = false;
}

else if (sensorValue > 300) {
```

```
    stageText = "EARLY";
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    alertTriggered = false;
}
else {
    stageText = "FRESH";
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    alertTriggered = false;
}

// Local Output Interface Logic (Display toggles every 2 seconds)
lcd.clear();
if (displayToggle) {
    lcd.setCursor(0, 0);
    lcd.print("Gas Value: ");
    lcd.print(sensorValue);
    lcd.setCursor(0, 1);
    lcd.print("Stage: ");
    lcd.print(stageText);
} else {
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(temperature, 1);
    lcd.print("°C");
    lcd.setCursor(0, 1);
    lcd.print("Humid: ");
    lcd.print(humidity, 1);
    lcd.print("%");
}
```

```
}

displayToggle = !displayToggle;
}

void setup() {
  Wire.begin(D2, D1);

  // Initialize Hardware Pin Registers
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  // Enforce safe startup power values
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  lcd.init();
  lcd.backlight();
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("System□ Booting...");
  lcd.setCursor(0, 1);
  lcd.print("Waking□ Sensors...");

  dht.begin();

  // Initialize network socket parameters
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, password);
  timer.setInterval(2000L, monitorSystem);
```

```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("System Online!");  
lcd.setCursor(0, 1);  
lcd.print("HARISH");  
delay(1500);  
}
```

```
void loop() {  
  Blynk.run();  
  timer.run();  
}
```

Key Implementation Features:

- **Blynk Integration:** Cloud connectivity with template ID and auth token for remote monitoring and alert triggering.
- **Sensor Fusion:** Real-time acquisition of MQ-135 gas concentration (analog pin A0) and DHT11 temperature-humidity data (pin D3).
- **4-Stage Classification:** Threshold-based spoilage stage determination:
 - **Fresh:** Gas ≤ 300 ppm
 - **Early Spoilage:** $300 < \text{Gas} \leq 500$ ppm
 - **Moderate Spoilage:** $500 < \text{Gas} \leq 750$ ppm
 - **Severe Spoilage:** Gas > 750 ppm
- **Visual Feedback:** Tri-color LED indicator (Green=Fresh/Early, Red=Moderate/Severe) and piezo buzzer alarm for severe conditions.
- **LCD Display:** 16x2 I2C LCD alternates between gas/stage and temperature/humidity readings every 2 seconds.
- **Asynchronous Monitoring:** BlynkTimer executes sensor monitoring at 2-second intervals without blocking the main loop.

- **Alert Mechanism:** Spoilage alerts are logged to Blynk cloud platform triggering push notifications and email alerts when severe spoilage is detected.

The appendix demonstrates important implementation modules used during the development of the proposed food spoilage detection system. These implementation details support the methodologies and experimental results discussed in the previous chapters.



BIBLIOGRAPHY

- [1] K. A. Nfor, T. P. T. Armand, K. P. I. Ismaylovna, M.-I. Joo, and H.-C. Kim, "An explainable cnn and vision transformer-based approach for real-time food recognition," *Nutrients*, vol. 17, no. 2, p. 362, 2025. DOI: 10.3390/nu17020362.
- [2] U. Amin, M. I. Shahzad, A. Shahzad, M. Shahzad, U. Khan, and Z. Mahmood, "Automatic fruits freshness classification using cnn and transfer learning," *Applied Sciences*, vol. 13, no. 14, p. 8087, 2023. DOI: 10.3390/app13148087.
- [3] BMC Medicine, "Food insecurity: A neglected public health issue requiring multi-sectoral action," *BMC Medicine*, vol. 21, p. 130, 2023. DOI: 10.1186/s12916-023-02845-3.
- [4] P. Kanupuru and N. V. U. Reddy, "A deep learning approach to detect the spoiled fruits," *ResearchGate*, vol. 10, 2022.
- [5] M. Mukhiddinov, A. Muminov, and J. Cho, "Improved classification approach for fruits and vegetables freshness based on deep learning," *Sensors*, vol. 22, no. 21, p. 8192, 2022. DOI: 10.3390/s22218192.
- [6] P. Ahmad, H. Jin, R. Alroobaea, S. Qamar, R. Zheng, and F. Alnajjar, "Mh-unet: A multi-scale hierarchical based architecture for medical image segmentation," *IEEE Access*, vol. 9, pp. 148 384–148 408, 2021. DOI: 10.1109/ACCESS.2021.3122543.
- [7] R. Arul, R. Alroobaea, S. Mechti, S. Rubaiee, M. Andejany, U. Tariq, *et al.*, "Intelligent data analytics in energy optimization for the internet of underwater things," *Soft Computing*, vol. 25, pp. 12 507–12 519, 2021. DOI: 10.1007/s00500-021-06002-x.
- [8] V. Bhole and A. Kumar, "A transfer learning-based approach to predict the shelf life of fruit," *Inteligencia Artificial*, vol. 24, no. 67, pp. 102–120, 2021. DOI: 10.4114/intartif.vol24iss67pp102-120.
- [9] C. Chauhan, A. Dhir, M. Akram, and J. Salo, "Food loss and waste in food supply chains: A systematic literature review and framework development approach," *Journal of Cleaner Production*, vol. 295, p. 126 438, 2021. DOI: 10.1016/j.jclepro.2021.126438.

- [10] C. Dhanamjayulu, U. Nizhal, P. Maddikunta, T. Gadekallu, C. Iwendi, and C. Wei, "Identification of malnutrition and prediction of bmi from facial images using real-time image processing and machine learning," *IET Image Processing*, pp. 1–12, 2021. DOI: 10.1049/ipr2.12222.
- [11] F. Santeramo, "Exploring the link among food loss, waste, and food security: What the research should focus on?" *Agriculture & Food Security*, vol. 10, no. 1, p. 26, 2021. DOI: 10.1186/s40066-021-00302-z.
- [12] S. A. Abbas and S. A. Shah, "Food spoilage detection using deep learning techniques: A review," *Journal of Food Science and Technology*, vol. 57, no. 4, pp. 1142–1155, 2020.
- [13] J. Anajemba, Y. Tang, C. Iwendi, A. Ohwokevw, G. Srivastava, and O. Jo, "Realizing efficient security and privacy in iot networks," *Sensors*, vol. 20, no. 9, p. 2609, 2020. DOI: 10.3390/s20092609.
- [14] M. T. Islam, "Plant disease detection using cnn model and image processing," *International Journal of Engineering Research & Technology*, vol. 9, no. 10, pp. 291–297, 2020.
- [15] G. Madhulatha and O. Ramadevi, "Recognition of plant diseases using convolutional neural network," in *Proceedings of the 2020 Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)*, IEEE, 2020, pp. 738–743.
- [16] S. Qiao, Q. Wang, J. Zhang, and Z. Pei, "Detection and classification of early decay on blueberry based on improved deep residual 3d convolutional neural network in hyperspectral images," *Scientific Programming*, vol. 2020, p. 8 895 875, 2020.
- [17] World Food Programme, *5 facts about food waste and hunger*, Available: World Food Programme, 2020.
- [18] D. Yadav and A. K. Yadav, "A novel convolutional neural network based model for recognition and classification of apple leaf diseases," *Traitement du Signal*, vol. 37, no. 6, 2020.
- [19] P. Jiang, Y. Chen, B. Liu, D. He, and C. Liang, "Real-time detection of apple leaf diseases using deep learning approach based on improved convolutional neural networks," *IEEE Access*, vol. 7, pp. 59 069–59 080, 2019.

- [20] M. S. Hossain, M. Al-Hammadi, and G. Muhammad, “Automatic fruit classification using deep learning for industrial applications,” *IEEE Transactions on Industrial Informatics*, vol. 15, no. 2, pp. 1027–1034, 2018.

